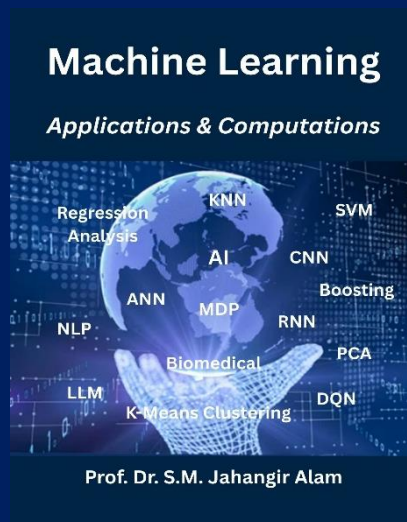


Lecture 4.1

Reinforcement Learning



Instructor Name:

Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Reinforcement Learning

- Reinforcement Learning is a type of **Machine Learning** where an **agent** learns to make decisions by interacting with an **environment** to maximize a **reward**. Instead of being told the “right answer” (like in supervised learning), the agent **learns by trial and error**.

Reinforcement Learning
= *Learning by trial & error with rewards.*

The goal
= *find the best policy to maximize long – term rewards.*

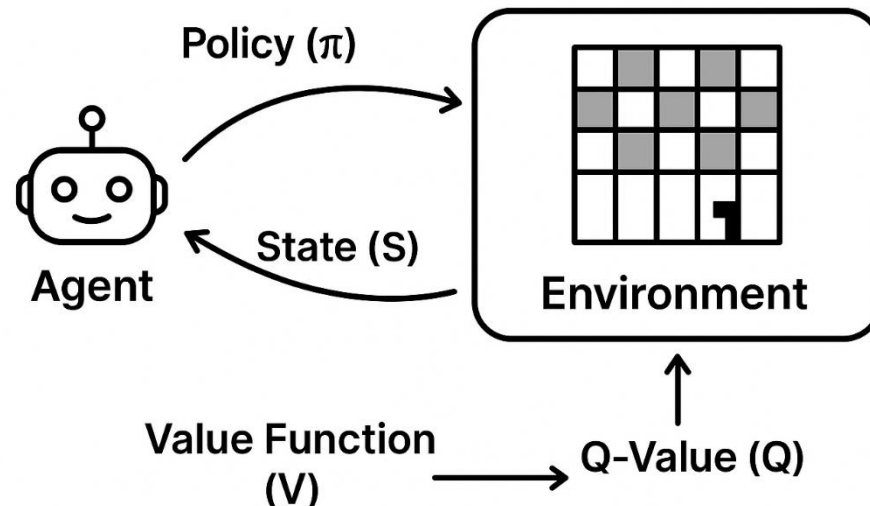
Key Elements of RL

Agent: The learner/decision-maker (e.g., robot, game bot).

Environment → The world in which the agent acts (e.g., chessboard, traffic system).

State (S) → Current situation of the environment (e.g., position of the robot).

Action (A) → Choices the agent can make (e.g., move left/right).



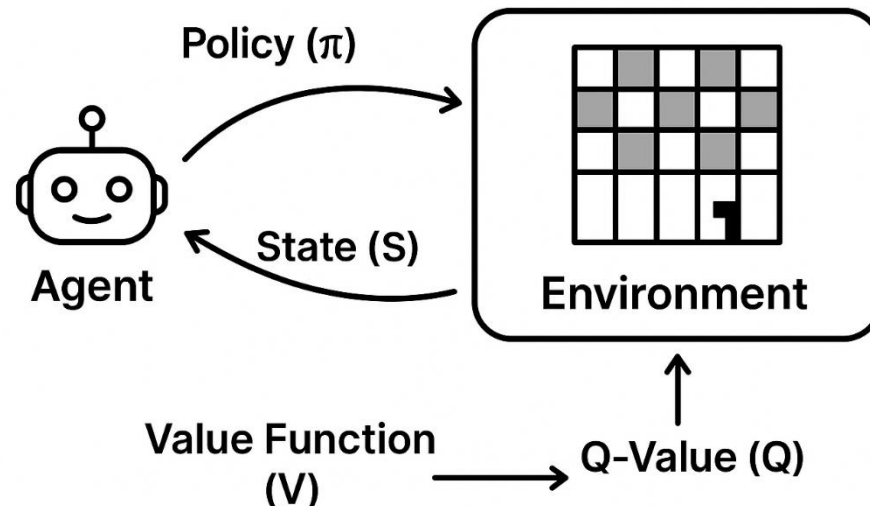
Key Elements of RL

Reward (R) → Feedback from the environment after an action (e.g., +1 for winning a move, -1 for losing).

Policy (π) → Strategy that defines how the agent chooses actions.

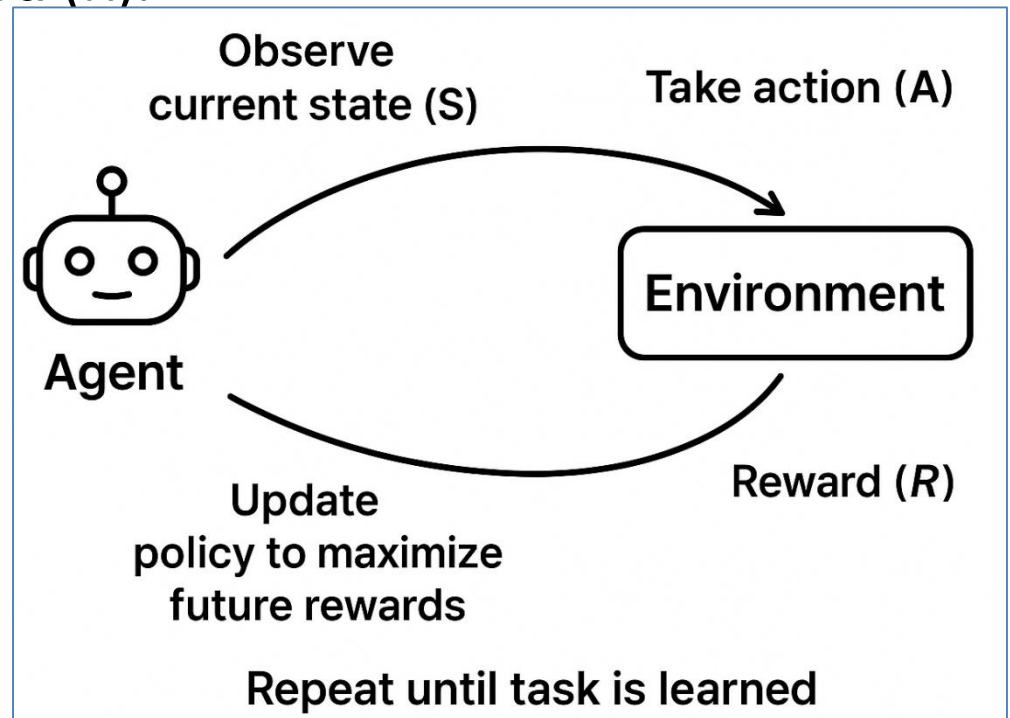
Value Function (V) → Expected long-term reward of being in a state.

Q-Value (Q) → Expected long-term reward of taking a specific action in a specific state.



RL Process (Loop)

- Agent observes the **current state** (S).
- Agent takes an **action** (A).
- Environment transitions to a **new state** (S').
- Environment gives a **reward** (R).
- Agent updates its **policy** to maximize future rewards.
- Repeat until the task is learned.



Types of Reinforcement Learning

1. **Model-Free:** Learns directly from experience (no model of environment).
 - **Q-Learning** (Q-learning teaches an agent **what to do** in each situation by learning from trial and error.)
 - **SARSA** (State-Action-Reward-State-Action)
2. **Model-Based:** Learns a model of environment and plans ahead.
 - Dyna-Q
 - Monte Carlo Tree Search (MCTS)
3. **Policy-Based:** Learns the policy directly.
 - REINFORCE, Proximal Policy Optimization (PPO).
4. **Actor–Critic:** Combines policy and value learning
 - Advantage Actor-Critic (A2C), Asynchronous Advantage Actor-Critic (A3C).

Applications of RL

- **Games:** Chess, Go, Atari, StarCraft (AlphaGo, AlphaStar), OpenAI Five (Dota2)
- **Healthcare:** Treatment planning, drug discovery
- **Self-driving cars:** navigation, control
- **Robotics:** walking, manipulation, Navigation, path planning
- **Autonomous Vehicles:** Driving policy optimization
- **Finance:** portfolio optimization, trading strategies
- Network optimization, recommendation systems
- **Recommendation Systems:** Personalized dynamic recommendations

Reinforcement Learning

Mathematical Framework

RL uses the **Markov Decision Process (MDP)** defined by:

$$\langle S, A, P, R, \gamma \rangle$$

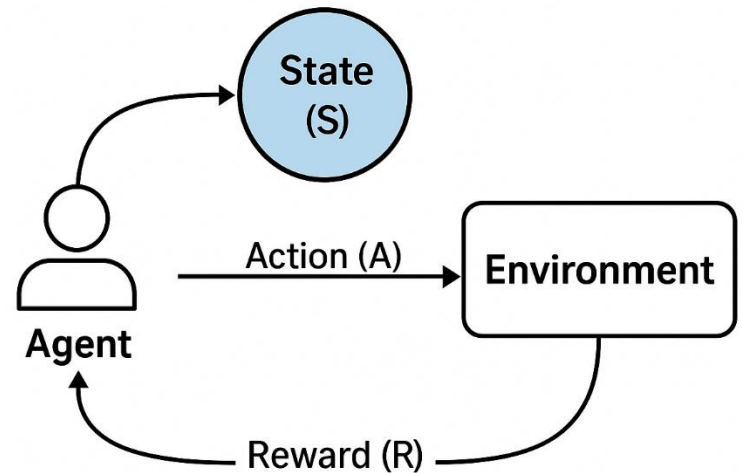
Where,

- S : set of possible states
- A : set of possible actions
- $P(s' | s, a)$: transition probability
- $R(s, a)$: reward function
- γ : discount factor ($0 \leq \gamma < 1$)

Goal:

Maximize total expected return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$

Reinforcement Learning



Reinforcement Learning

4.1##A simple Reinforcement Learning (RL) using the Q-learning algorithm, one of the foundational RL methods.

- Task: A robot learns to navigate a 1D grid and reach the goal
- Environment: 1D Grid World
- We have 5 positions (states):
- [Start] - 1 - 2 - 3 - [Goal]
- States: 0 to 4
- Actions: Left (L), Right (R)
- Goal = State 4
- Goal: Reach State 4 and get a reward of +10, all other moves get 0

Reinforcement Learning

Step 1: Initialize Q-Table Step 2: Define Parameters

State	Action	Q-Value
0	R	0
1	L	0
1	R	0
2	L	0
2	R	0
3	L	0
3	R	0

(Q-values for impossible actions like L from state 0 or R from state 4 are not included.)

Learning rate $\alpha=0.5$

Discount factor $\gamma=0.9$

Initial state: 0

Goal state: 4

Q-update rule:

$Q(s, a)$

$$\leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Or,

$$Q(s, a) = E \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

Reinforcement Learning

- **Step 3: Run One Episode**
- Start at $0 \rightarrow R \rightarrow 1 \rightarrow R \rightarrow 2 \rightarrow R \rightarrow 3 \rightarrow R \rightarrow 4$ (Goal)
- Action 1: From state $0 \rightarrow R \rightarrow$ state 1
- Old $Q(0,R) = 0$
- Reward = 0
- Next $\max Q(1,*) = 0$
- $Q(0,R) = 0 + 0.5[0 + 0.9 \cdot 0 - 0] = 0$
- (No update — reward is 0, no value in future yet.)

Reinforcement Learning

- Action 2: $1 \rightarrow R \rightarrow 2$
- Same as above $\rightarrow Q(1,R)$ stays 0
- Action 3: $2 \rightarrow R \rightarrow 3$
- Same $\rightarrow Q(2,R) = 0$
- Action 4: $3 \rightarrow R \rightarrow 4$ (Goal)
- Reward = +10
- Next state is terminal, so $\max Q = 0$
- $Q(3,R) = 0 + 0.5 [10 + 0 - 0] = 5$
- Updated:
- $Q(3, R) = 5$

Reinforcement Learning

Final Q-table after 1 episode:

Next episodes will back-propagate value:

- $Q(2, R)$ will increase because it leads to state 3 with $Q=5$
- Then $Q(1, R) \rightarrow Q(0, R)$

State	Action	Q-Value
0	R	0
1	R	0
2	R	0
3	R	5

Summary:

- Agent starts with zero knowledge
- Learns from trial and reward
- Uses Q-learning to update future value of actions

With repeated episodes, Q-values converge to the optimal policy

Reinforcement Learning

4.2##Let's go through a Reinforcement Learning (RL) basics with a very simple setup.

Simple Example: Grid World

Imagine a tiny 2x2 grid where an agent wants to reach a **goal state**.

States (positions)	Description
S1 (top-left)	Start state
S2 (top-right)	Normal
S3 (bottom-left)	Normal
S4 (bottom-right)	Goal (reward +10)

Reinforcement Learning

Agent actions:

- Up, Down, Left, Right
- If agent hits grid boundary, stays in the same state.

Step 1: Define states, actions, rewards

State	Actions possible	Reward
S1	Right (to S2), Down (to S3)	0
S2	Left (to S1), Down (to S4)	0
S3	Up (to S1), Right (to S4)	0
S4	— (goal)	+10 (terminal state)

Reinforcement Learning

Step 2: Q-learning setup

- We want to learn the **Q-values** $Q(s, a)$ that estimate expected rewards for taking action a in state s .
- Initialize all $Q(s, a) = 0$.

Step 3: Sample episode

- Assume learning rate $\alpha = 0.5$ and discount factor $\gamma = 0.9$.

Reinforcement Learning

Episode 1

Start at **S1**:

- Take action **Right** to **S2**, reward = 0.

- Update:

$$\begin{aligned} Q(S1, Right) \\ &= Q(S1, Right) + \alpha[r + \gamma \max_a Q(S2, a) - Q(S1, Right)] \\ &= 0 + 0.5[0 + 0.9 \times 0 - 0] = 0 \end{aligned}$$

At **S2**:

- Take action **Down** to **S4** (goal), reward = 10.

- Update:

$$Q(S2, Down) = 0 + 0.5[10 + 0.9 \times 0 - 0] = 0.5 \times 10 = 5$$

At **S4**: terminal, no action.

Reinforcement Learning

Step 4: Episode 2

Start at **S1** again:

- Take **Right** to **S2**, reward = 0.

- Update:

$$\begin{aligned} Q(S1, Right) \\ = 0 + 0.5[0 + 0.9 \times \max(Q(S2, Left), Q(S2, Down)) - 0] \end{aligned}$$

We know:

- $Q(S2, Left) = 0$
- $Q(S2, Down) = 5$

Thus:

$$Q(S1, Right) = 0.5 \times (0 + 0.9 \times 5) = 0.5 \times 4.5 = 2.25$$

Reinforcement Learning

At S2:

Take **Down** to **S4**, reward = 10.

Update:

$$Q(S2, Down) = 5 + 0.5[10 + 0.9 \times 0 - 5] = 5 + 0.5 \times 5 = 7.5$$

Current Q-table snapshot:

State	Action	Q-value
S1	Right	2.25
S1	Down	0
S2	Left	0
S2	Down	7.5

Summary:

- Agent learns expected rewards for actions.
- Q-values updated with new experience.
- Over time, agent prefers actions leading to higher reward (goal).

Earth Quake Detection Using Classification

Goal: Detect if a seismic signal indicates an earthquake.

Step 1: Data from a Seismic Sensor

- We collect ground acceleration (in units of m/s^2) over time:

Time (s)	Ground Acceleration (m/s^2)
0.0	0.001
0.1	0.003
0.2	0.002
0.3	0.010
0.4	0.080 ←
0.5	0.120 ←
0.6	0.150 ←
0.7	0.020
0.8	0.003

Step 2: Set a Threshold

Typical background noise: $\leq 0.01 \text{ m/s}^2$
Let's say, Earthquake threshold = 0.05 m/s^2

Earth Quake Detection Using Classification

Goal: Detect if a seismic signal indicates an earthquake.

Step 1: Data from a Seismic Sensor

- We collect ground acceleration (in units of m/s^2) over time:

Time (s)	Ground Acceleration (m/s^2)
0.0	0.001
0.1	0.003
0.2	0.002
0.3	0.010
0.4	0.080 ←
0.5	0.120 ←
0.6	0.150 ←
0.7	0.020
0.8	0.003

Step 3: Apply Threshold Rule

Check which points exceed threshold:

- At 0.4s: 0.080 → above threshold
 - At 0.5s: 0.120 → above threshold
 - At 0.6s: 0.150 → above threshold
- Sustained 3 readings over threshold →
Earthquake detected

Earth Quake Detection Using Classification

Step 4: Train a Simple Classifier (Supervised ML)

Let's say we use 2 features:

- Peak Acceleration
- Duration Above Threshold

Example dataset:

Event	Peak Accel	Duration Above Threshold (s)	Earthquake (Label)
A	0.15	0.3	1
B	0.03	0	0
C	0.08	0.2	1
D	0.01	0	0

Earth Quake Detection Using Classification

- Simple rule:

If peak > 0.05 and duration $> 0.2s \rightarrow$ Earthquake

For our data:

- Peak = 0.15
- Duration above threshold = 0.3s
 $\rightarrow$ Earthquake

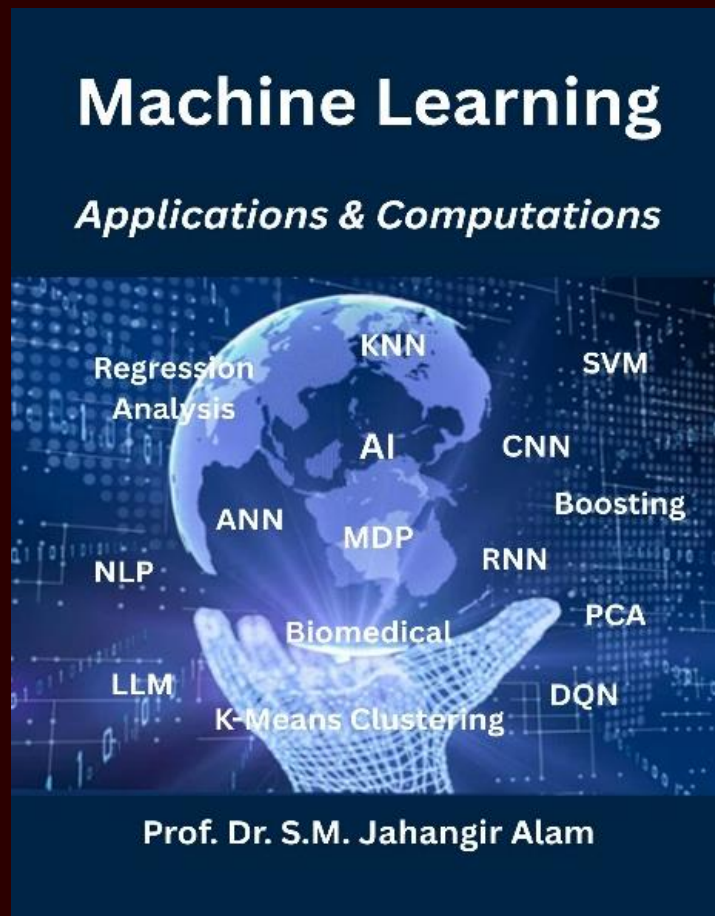
Final Conclusion

Earthquake Detected

- Peak ground acceleration: 0.15 m/s^2
- Duration over threshold: 0.3 seconds
- Decision: Yes, this is a seismic event

Lecture 4.2

Recommendation System



Instructor Name:

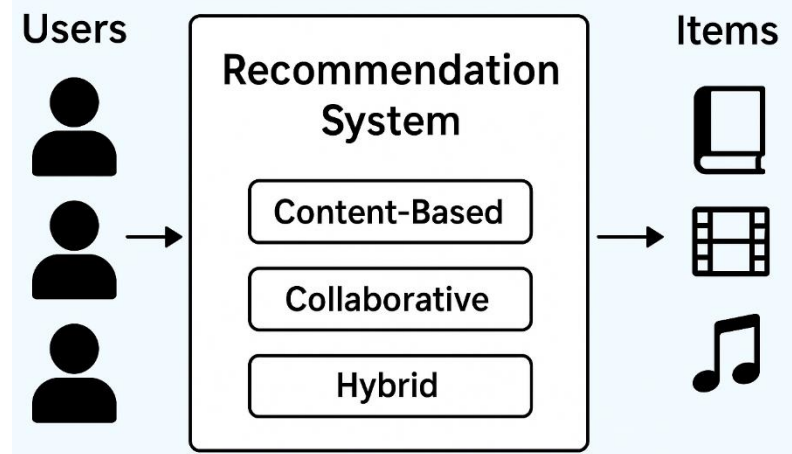
Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

Recommendation System

- A **Recommendation System (RS)** suggests relevant items to users based on their preferences and behavior. A Recommendation System learns **patterns in user preferences** (via content, behavior, or latent factors) to suggest **the right items to the right users**.

- **Examples:**
- Netflix → recommends movies
- Amazon → recommends products
- Spotify → recommends songs
- YouTube → Suggest videos



Recommendation System

Applications

Domain	Example
E-commerce	Product recommendation (Amazon, Flipkart)
Streaming Platforms	Movie/song recommendations (Netflix, Spotify)
Social media	TikTok, Friend suggestion, content recommendation (Facebook, Instagram)
Education	Recommend courses or learning materials
Healthcare	Suggest health interventions or medication adherence, Drug or treatment recommendations.

Recommendation System

- **Modern Recommendation Systems:**
- Deep Learning-based: Neural Collaborative Filtering, Autoencoders.
- Sequence Models: RNN, Transformers (for next-item prediction).
- Graph Neural Networks (GNN): Model user-item relationships as a graph.

Types of Recommendation Systems

- 1. Content-Based Filtering**
- 2. Collaborative Filtering**
- 3. Matrix Factorization (Latent Factors)**
- 4. Hybrid Methods**
- 5. Deep Learning Based Recommendation**

Types of Recommendation Systems

1. Content-Based Filtering

- Uses features of items and user profile.
- Example: If you liked ***Inception*** (Sci-Fi, Thriller), the system recommends other Sci-Fi thrillers.
- Method: $\text{cosine similarity}(item_i, item_j) = \cos(\theta) = \frac{A \cdot B}{||A|| \cdot ||B||}$

2. Collaborative Filtering

- Uses user-item interaction matrix (ratings, clicks, purchases).
- Two approaches:
 - User-based CF → Find similar users to recommend items.
 - Item-based CF → Find similar items and recommend them.
 - Example: If User A and User B both liked *Movie X*, and User A also liked *Movie Y*, then recommend *Movie Y* to User B.

Types of Recommendation Systems

3. Matrix Factorization (Latent Factors)

- Decomposes the user-item rating matrix into latent vectors (SVD, ALS).
- $R \approx U \cdot V^T$
- Netflix Prize (2009) used this technique.

4. Hybrid Methods

- Combine content-based + collaborative filtering.
- Example: YouTube → combines video features + user behavior.

Types of Recommendation Systems

5. Deep Learning Based Recommendation

- Use **neural networks** to model complex patterns in user-item interactions.
- Examples:
 - **Autoencoders** → reconstruct user-item interactions
 - **Neural collaborative filtering (NCF)**
 - **Sequence-based models** → RNNs or Transformers for session-based recommendations

User-Item Ratings and Collaborative Filtering

4.19##A simple Recommendation System using User-Item Ratings and Collaborative Filtering with Cosine Similarity.

Setup: Users and Items ratings matrix:

Goal: Predict User A's rating for Item 3 (marked as '?').

User \ Item	Item 1	Item 2	Item 3
User A	5	3	?
User B	4	?	2
User C	?	4	5

Step 1: Compute similarity between users

Similarity between User A and User B

Common items rated: Item 1

- User A's rating vector: [5]
- User B's rating vector: [4]

$$\text{cosine similarity} = \frac{5 \times 4}{\sqrt{5^2} \times \sqrt{4^2}} = 1$$

Similarity between User A and User C

Common items rated: Item 2

- User A: 3
- User C: 4

$$\text{cosine similarity} = \frac{3 \times 4}{\sqrt{3^2} \times \sqrt{4^2}} = 1$$

User-Item Ratings and Collaborative Filtering

Step 2: Predict rating for User A on Item 3

Using weighted average based on similarities and other users' ratings on Item 3:

$$\hat{r}_{A,3} = \frac{\sum_{u \in \{B,C\}} \text{sim}(A, u) \times r_{u,3}}{\sum_{u \in \{B,C\}} \text{sim}(A, u)}$$

User \ Item	Item 1	Item 2	Item 3
User A	5	3	?
User B	4	?	2
User C	?	4	5

Where: $r_{B,3} = 2$, $r_{C,3} = 5$, $\text{sim}(A, B) = 1$, $\text{sim}(A, C) = 1$.

Predicted rating:

$$\hat{r}_{A,3} = \frac{1 \times 2 + 1 \times 5}{1 + 1} = 3.5$$

Result: Predicted rating for User A on Item 3 is 3.5

User-Item Ratings and Collaborative Filtering

Summary:

- Compute similarity between users on commonly rated items.
- Predict missing rating by weighted average of other users' ratings.
- Simple collaborative filtering approach.

Recipe Recommendation System

4.20##A Recipe Recommendation System demonstration of how such a system works useful for learning, teaching, or interview settings.

Objective:

Given user preferences or available ingredients, recommend a relevant recipe.

Step 1: Input — Ingredients Available

Let's say the user has:

["tomato", "onion", "garlic", "pasta", "basil", "cheese"]

Recipe Recommendation System

Step 2: Recipe Dataset (Small Example)

Recipe ID	Recipe Name	Ingredients
R1	Tomato-Pasta	tomato, onion, garlic, pasta, olive oil
R2	Garlic-Bread	garlic, butter, bread
R3	Caprese-Salad	tomato, basil, mozzarella
R4	Fried-Rice	rice, soy sauce, onion, egg

["tomato", "onion", "garlic", "pasta", "basil", "cheese"]

Recipe Recommendation System

Step 3: Matching Score

We'll use **Jaccard Similarity**:

$$Score = \frac{| \text{Ingredients in common} |}{| \text{Total unique ingredients} |}$$

R1: Tomato Pasta

Overlap = 4 → (tomato, onion, garlic, pasta)

Total unique = 6 → (tomato, onion, garlic, pasta, olive oil, cheese)

$$Score = \frac{4}{6} \approx 0.67$$

R2: Garlic Bread

Overlap = 1 → (garlic)

Total = 5 → (garlic, butter, bread, cheese, onion)

$$Score = \frac{1}{5} = 0.2$$

Recipe Recommendation System

R3: Caprese Salad

Overlap = 2 → (tomato, basil)

Total = 5 → (tomato, basil, mozzarella, cheese, onion)

$$Score = \frac{2}{5} = 0.4$$

R4: Fried Rice

Overlap = 1 → (onion)

Total = 6 → (rice, soy sauce, onion, egg, garlic, cheese)

$$Score = \frac{1}{6} \approx 0.17$$

Step 4: Recommend Top Recipe

Final Recommendation: Recommended Recipe is Tomato Pasta.

Recipe Name	Score
Tomato Pasta	0.67
Caprese Salad	0.4
Garlic Bread	0.2
Fried Rice	0.17

Recipe Recommendation System

Summary:

Step	Description
Input	User's available ingredients
Dataset	Recipes with ingredient lists
Matching	Use Jaccard similarity
Output	Recommend recipe with highest similarity

Ted Talks Recommendation System with ML

4.21##A simple Ted Talks Recommendation System using a basic content-based filtering approach with machine learning concepts.

Objective: Recommend Ted Talks similar to a user's liked talks based on talk features (like topic keywords).

Step 1: Define Ted Talks Dataset

Talk ID	Title	Keywords
1	"AI and Future"	["AI", "technology", "future"]
2	"Health and Wellness"	["health", "wellness", "fitness"]
3	"Machine Learning"	["AI", "machine learning", "data"]
4	"Mindfulness"	["mindfulness", "health", "meditation"]
5	"Data Science"	["data", "statistics", "machine learning"]

Ted Talks Recommendation System with ML

Step 2: User Profile (talks user liked):

User liked Talk 1 and Talk 3,

- Talk 1 Keywords: ["AI", "technology", "future"]
- Talk 3 Keywords: ["AI", "machine learning", "data"]

Step 3: Feature Representation (Bag of Words vector)

Create a vocabulary from all keywords:

Vocabulary

= {AI, technology, future, health, wellness, fitness, machine learning, data, mindfulness, meditation, statistics}

Keyword	Index
AI	0
Technology	1
future	2
health	3
wellness	4
fitness	5
machine learning	6
data	7
mindfulness	8
meditation	9
statistics	10

Ted Talks Recommendation System with ML

Step 4: Vectorize Talks

Each talk is represented as an 11-dim vector with 1 if keyword present, else 0.

Talk ID	Vector
1	[1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]
2	[0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0]
3	[1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0]
4	[0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0]
5	[0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1]

Step 5: Create User Profile Vector

User profile = average of liked talks' vectors (Talk 1 and Talk 3):

$$\begin{aligned}\vec{u} &= \frac{\vec{t}_1 + \vec{t}_3}{2} = \frac{[1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0] + [1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0]}{2} \\ &= [1, 0.5, 0.5, 0, 0, 0, 0.5, 0.5, 0, 0, 0]\end{aligned}$$

Ted Talks Recommendation System with ML

Step 6: Compute Similarity (Cosine similarity) Between User Profile and Other Talks

Cosine similarity formula:

$$\text{sim}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \times \|\vec{v}\|}$$

Calculate similarity for Talk 2, 4, 5:

Talk 2 vector: [0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0]

$$\vec{u} \cdot \vec{t}_2 = 0$$

similarity = 0

Talk 4 vector: [0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0]

$$\vec{u} \cdot \vec{t}_4 = 0$$

Similarity = 0

Ted Talks Recommendation System with ML

$$\vec{u} = \frac{\vec{t}_1 + \vec{t}_3}{2} = [1, 0.5, 0.5, 0, 0, 0, 0.5, 0.5, 0, 0, 0]$$

Talk 5 vector: [0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1]

$$\vec{u} \cdot \vec{t}_5 = 0.5 \times 1 + 0.5 \times 1 + 0 \times 1 = 1.0$$

Calculate norms:

$$\cdot \quad \|\vec{u}\| = \sqrt{[1, 0.5, 0.5, 0, 0, 0, 0.5, 0.5, 0, 0, 0]} = \sqrt{2} \approx 1.414$$

$$\cdot \quad \|\vec{t}_5\| = \sqrt{[0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1]} = \sqrt{3} \approx 1.732$$

Similarity:

$$sim(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \times \|\vec{v}\|} = \frac{1.0}{1.414 \times 1.732} \approx 0.408$$

Ted Talks Recommendation System with ML

Step 7: Recommend

Sort talks by similarity score:

Talk ID	Similarity
5	0.408
2	0
4	0

Nex Recommended **Talk 5: "Data Science"**.

Summary:

Step	Description
User liked talks	Talk 1, Talk 3
User profile vector	Average of liked talks vectors
Similarity computed	Cosine similarity with other talks
Next Recommended talk	Talk 5 ("Data Science")

Movie Recommendation Based on Emotion

4.22##Consider a simple Movie Recommendation Based on Emotion.

Objective: Recommend movies to a user based on detected or declared emotion.

Step 1: Define Movies and Associated Emotions

Movie ID	Movie Title	Associated Emotion(s)
1	"The Pursuit of Happiness"	Happy, Inspirational
2	"Inside Out"	Happy, Sad
3	"The Notebook"	Romantic, Sad
4	"Joker"	Sad, Angry
5	"Deadpool"	Happy, Funny
6	"Gladiator"	Angry, Inspirational

Movie Recommendation Based on Emotion

Step 2: User's Current Emotion

Suppose the user's detected emotion is **Sad**.

Step 3: Recommend Movies Matching Emotion

Select movies associated with **Sad**:

- Movie 2: "Inside Out" (Happy, Sad)
- Movie 3: "The Notebook" (Romantic, Sad)
- Movie 4: "Joker" (Sad, Angry)

Step 4: Rank Recommendations (optional)

If we have user preferences or ratings, we can rank. Here, assume all equal. Recommend movies:

1. "Inside Out"
2. "The Notebook"
3. "Joker"

Movie Recommendation Based on Emotion

Step 5: If multiple emotions detected, combine

Suppose detected emotions: **Sad** and **Happy**

Movies matching either emotion:

- Sad: 2, 3, 4
- Happy: 1, 2, 5

Sad U Happy: 1, 2, 3, 4, 5

Step 6: Score by number of matching emotions

Count how many emotions match per movie:

Movie ID	Title	Matches	Score
1	The Pursuit of Happiness	Happy (1)	1
2	Inside Out	Happy, Sad (2)	2
3	The Notebook	Sad (1)	1
4	Joker	Sad (1)	1
5	Deadpool	Happy (1)	1

Movie Recommendation Based on Emotion

Recommend sorted by score:

1. "Inside Out" (score 2)
2. Others (score 1)

Summary:

Step	Action	Result
User emotion	Sad	Select movies with Sad
Recommendation	Movies 2, 3, 4	"Inside Out", "Notebook", "Joker"
Multi-emotion	Sad & Happy	Recommend movies 1,2,3,4,5
Scoring	Count matched emotions	Rank movies by score

Food Recognition Selenium using Caloriemama API

4.23##Demonstrating how you might build a simple Food Recognition automation using Selenium and CalorieMama API. This example explains the workflow and key steps you'd do manually or programmatically for learning or interview purposes.

Objective:

- Use **Selenium** to automate food image upload on CalorieMama (or similar API)
- Use **CalorieMama API** to get food recognition results (food name, calories, nutrients)



Step 1: Selenium Automation to Upload Image

Scenario: Automate browser to upload a food image for recognition.

Input: Image file: "pizza.jpg"

Food Recognition Selenium using Caloriemama API

Manual steps automated by Selenium:

Action	Explanation
Open CalorieMama web page	Navigate to the food recognition page
Locate file upload button	Find the <code><input type="file"></code> element
Upload "pizza.jpg"	Selenium sends file path to upload input
Submit image for recognition	Click "Analyze" or equivalent button
Wait for results	Wait for API/webpage to return results

Food Recognition Selenium using Caloriemama API

Step 2: CalorieMama API Request

Alternatively, call API directly with image data:

- API Endpoint: <https://api.caloriemama.ai/v1/food/recognition>
- **Headers:**
 - Authorization: Bearer YOUR_API_KEY
 - Content-Type: multipart/form-data
- **Payload:**
 - Image file in binary form (e.g., pizza.jpg)

Food Recognition Selenium using Caloriemama API

Step 3: Sample API Response (Simplified)

```
{ "foods": [  
  { "name": "Pizza",  
    "calories": 285,  
    "serving_size": "1 slice",  
    "nutrients": {  
      "protein": "12g",  
      "fat": "10g",  
      "carbohydrates": "35g"  
    }  
  }  
]
```

Food Recognition Selenium using Caloriemama API

Step 4: Processing the Response

- Extract food name: **Pizza**
- Calories: **285 kcal per slice**
- Nutritional info: protein, fat, carbs

Summary:

Step 5: Present Results

Attribute	Value
Food Name	Pizza
Calories	285 kcal
Serving Size	1 slice
Protein	12 g
Fat	10 g
Carbohydrates	35 g

Step	Action
Selenium Automation	Open page → Upload image → Submit for recognition
API Call	Send image to CalorieMama API with proper headers
Receive Response	Parse JSON with food and nutrient info
Display Results	Show food name, calories, macros

Thank You